

Spare Life iOS Stage 3 综合优化报告

1. 报告目标

这份文档把 Stage 3 已完成的 30 个改进方向整合成一份统一总报告，作为 Stage 3 架构优化的综合说明书。

它不是替代原始 30 篇研究文档，而是做三件事：

1. 把 30 个方向放回同一个全局架构里，避免只见单点、不见体系。
2. 把每一项的现状、问题、改进目标、执行动作和预期结果整理成统一口径。
3. 给后续 Stage 4 或实现期提供一份更适合管理、复盘、沟通和导出 PDF 的总材料。

解释顺序保持不变：

1. 运行代码优先。
2. 本文是整合报告，不是新的需求源。
3. 单项详细证据和论证仍以原始研究文档为准。

2. Stage 3 总结论

Stage 3 的核心不是“多做了 30 个点”，而是把这个仓库从“产品设想、运行时代码、支持代码、插件代码、验证日志混在一起”的状态，收敛成了一个更可治理的仓库形态。

这 30 个方向最终围绕 5 条主线展开：

1. 真相治理：

把命名、文档层级、追踪方式、配置来源和验证证据统一下来。

2. 应用壳层治理：

把 5-tab 应用壳、共享 UI 层、导航呈现和内部工具面重新定义边界。

3. Xianxia 与 Masters 运行时治理：

把最接近真实链路的两个模块拆清楚，防止大文件和“半 live 半 fallback”状态继续失控。

4. EarnSocial、Messages、Profile 一致性治理：

把“真正运行的页面路径”和“未接线的大型经验模型/辅助代码”分开，收敛成单一运行真相。

5. 仓库边界、验证与自动化治理：

把 support code、plugin、package boundary、验证矩阵、fixture 和自动化本身都纳入统一规则。

最终输出结果是：

- ‘Docs/Stage_3_AR_Blueprint.md’ 作为 30 项权威完成清单
- ‘Docs/Stage_3_Codebase_Audit.md’ 作为代码与文档关系审计

- `Docs/researches/Stage\_3\_AR/` 下 30 篇研究文档
- `.ops/stage3\_ar/` 与 `.cron/` 下的自动化执行框架

3. 当前仓库总判断

在 Stage 3 完成后，可以对这个仓库做一个更清晰的总判断：

1. 这是一个“SwiftUI 应用壳 + 部分真实 gateway 模块 + 大量 support/mock/plugin 共仓”的混合仓库。
2. Xianxia 和 Masters 是最接近真实运行链路的模块。
3. EarnSocial、Messages、MyProfile 在根页层仍不同程度依赖 mock 或双路径实现。
4. spare-life-ios-app/Services/*.mjs、LocalBackend/、Domain/UseCases/*.mjs、spare-life-openclaw-plugin/ 对架构有重要影响，但并不等于全部进入 SpareLifeCore 的实际 Swift runtime。
5. Stage 3 的价值就在于：不再让“存在代码”自动等于“已接线能力”，也不再让“旧文档写过”自动等于“运行时真相”。

4. 五大板块总览

4.1 Section 1：真相治理与文档治理

这一组解决的是“仓库究竟以什么为准”的问题。它不直接改 UI，也不直接改业务，但它决定后续所有优化是否会继续跑偏。

4.2 Section 2：应用壳层与共享 UI

这一组解决的是“5-tab 应用怎么作为一个整体稳定运行”的问题。它把 tab、路由、共享 page chrome、shared UI 和内部工具面放回同一框架。

4.3 Section 3：Xianxia 与 Masters 运行时主链路

这一组解决的是“最重要的 live-ish 模块能否继续扩展”的问题。重点不是继续堆功能，而是先把文件、状态机、分页、ASR、fallback 和证据结构治理好。

4.4 Section 4：EarnSocial、Messages、Profile 的一致性

这一组解决的是“哪些代码是真的运行路径，哪些只是设计/支持代码”的问题。它的本质是收敛双路径和 mock/live 混杂。

4.5 Section 5：仓库边界、验证、fixture 与自动化

这一组解决的是“仓库规模扩大之后，怎样保证边界不乱、验证不飘、自动化不变成新的混乱源”的问题。

5. Section 1 : Source Of Truth And Documentation Governance

S1-01 统一产品名、UI 名、代码名、接口名词典

现状：

仓库里同一模块存在 闲人 / 咸虾 / 闲虾 / xianxia 以及 大师 / 闲聊 / masters 多套叫法，代码、UI 文案、旧文档和接口语义没有完全统一。

问题：

命名漂移会让代码审计、文档追踪、接口治理和后续自动化都变得脆弱，尤其会让“一个问题究竟指向哪个模块”变得模糊。

改进目标：

建立“产品名、UI 展示名、代码域名、接口名”四层词典，明确每个词该出现在哪里，不允许继续自由漂移。

建议动作：

统一用词典文件约束 tab 名、模块名、文档引用名、schema 名和日志名；旧名只作为兼容 alias 出现，不再作为主名。

预期结果：

后续任何变更都能先回答“这个词在仓库里属于哪一层”，从而减少认知成本和错误归类。

来源：

[S1-01 原始研究](researches/Stage_3_AR/section_1_truth/01-canonical-module-lexicon.md)

S1-02 重构文档分层

现状：

旧 Stage 文档既写需求、又写实现、又写验证、又写实时运行日志，导致文档既长、又难 diff、也难继续自动化维护。

问题：

一份文档承担过多角色时，任何修改都会伤到别的语义层。需求一改，验证记录也被改；验证记录一追加，权威需求源又被污染。

改进目标：

明确区分：

1. 产品蓝图
2. 实施蓝图
3. 执行镜像

- 4. 验证证据
- 5. 运行日志
- 6. 研究报告

建议动作：

把权威 requirement、可勾选 checklist、结构化验证结果和时间戳日志拆开存放，并在引用层建立稳定链接。

预期结果：

文档更新不再相互污染，自动化 worker 也能安全地只处理自己负责的文档层。

来源：

[S1-02 原始研究](researches/Stage_3_AR/section_1_truth/02-document-stratification.md)

S1-03 统一代码到文档的追踪格式

现状：

代码注释里已经有很多 blueprint 引用，但格式不统一，粒度也不一致。

问题：

如果代码到文档的追踪方式不统一，后面做审计、自动补全、验证回链时都要做高噪音清洗。

改进目标：

建立一套统一 trace 规范，让文件头注释、蓝图条目、验证记录、研究文档都能稳定互相引用。

建议动作：

统一 trace ID、文件头部格式、文档中的来源/影响/结果结构，以及研究文档到代码文件的引用方式。

预期结果：

后续任何人都能从一个条目反查到代码、从代码再反查到研究和验证。

来源：

[S1-03 原始研究](researches/Stage_3_AR/section_1_truth/03-code-to-doc-traceability.md)

S1-04 输出仓库级运行真相地图

现状：

仓库内同时有 Swift runtime、support.mjs、local backend、plugin workspace 和大量 docs。

问题：

如果没有统一的运行真相地图，容易把“存在的代码”误读为“已进入运行主路径的代码”。

改进目标：

用一份仓库地图明确标识：

1. 哪些是运行时代码
2. 哪些是 support code
3. 哪些是 prototype 或 future lane
4. 哪些是 plugin runtime
5. 哪些只是 docs-only

建议动作：

把目录边界、作用、是否 shipped、与 app 的关系、与验证的关系写清楚。

预期结果：

后续做架构判断时，不再把 support 层和 app runtime 混为一谈。

来源：

[S1-04 原始研究](researches/Stage_3_AR/section_1_truth/04-runtime-truth-map.md)

S1-05 统一配置来源登记方式

现状：

当前仓库已有多种配置来源：env、UserDefaults、keychain、本地 assets、默认常量和旧文档假设。

问题：

配置解析链如果不统一，问题定位会很慢，安全边界也会模糊。例如 live endpoint 未注入时，到底该去查 env、defaults 还是 keychain，很容易混乱。

改进目标：

建立“配置来源登记册”，为每一类配置定义优先级、来源、可见性和面向用户的诊断方式。

建议动作：

把 endpoint、auth、model、local asset、runtime toggle 这几类配置全部纳入统一 registry 思路。

预期结果：

配置诊断不再依赖口口相传，live blocker 能更快定位，安全字段的暴露策略也更可控。

来源：

[S1-05 原始研究](researches/Stage_3_AR/section_1_truth/05-configuration-source-registry.md)

S1-06 重构 Stage 级验证证据格式

现状：

旧 Stage 文档里混有大量时间戳 rerun 记录，验证信息缺少统一结构。

问题：

同样的验证会在文档里重复几十次，最终既影响可读性，也让“当前有效结论”被淹没。

改进目标：

把 Stage 验证拆成：

1. 权威需求
2. 状态镜像
3. 结构化验证结果
4. 运行日志

建议动作：

把“最新有效结论”和“时间序列运行日志”分开保存，避免下一阶段继续膨胀成操作日志仓库。

预期结果：

任何人看验证材料时，先看到结论，再决定是否深入看日志细节。

来源：

[S1-06 原始研究](researches/Stage_3_AR/section_1_truth/06-validation-evidence-format.md)

6. Section 2：App Shell And Shared UI Architecture

S2-01 统一根应用壳层元数据

现状：

5 个 tab 已成型，但根壳层的命名、analytics key、deep-link key、UI label、router target 没有形成同一元数据表。

问题：

随着模块增多，tab 信息会分散到多个文件、多个判断分支和多个常量里，导致壳层扩展变得昂贵。

改进目标：

建立一个统一的 app shell metadata 层，让 tab 身份、展示文案、路由键、图标、分析键同源。

建议动作：

为 tab 建立单点 metadata 定义，并限制页面层直接写死这些标识。

预期结果：

应用壳更像“有 schema 的 shell”，而不是“几个页面拼出来的 tab 容器”。

来源：

[S2-01 原始研究](researches/Stage_3_AR/section_2_shell_ui/01-app-shell-metadata.md)

S2-02 梳理导航与页面呈现契约

现状：

当前使用 TabView、NavigationStack、fullScreenCover 和少量 router 混合完成页面呈现。

问题：

这种方式在页面少的时候还能工作，但一旦跨 tab 跳详情、从首页直接进聊天、或者需要恢复会话状态，呈现逻辑会变复杂。

改进目标：

定义根路由、tab 内路由、modal 呈现、全屏聊天页呈现、返回行为和状态恢复之间的统一契约。

建议动作：

把“谁负责 present”“谁负责 dismiss”“何时保留 tab 状态”“何时进入全屏路径”标准化。

预期结果：

跨页面跳转不再依赖临时写法，后续增加消息、会诊、内部工具页时也更稳。

来源：

[S2-02 原始研究](researches/Stage_3_AR/section_2_shell_ui/02-navigation-presentation-contract.md)

S2-03 抽象首页通用 page chrome

现状：

多个首页已经都具备 header、搜索、筛选、轻模块和主内容区，但大量 page chrome 仍是模块内重复实现。

问题：

重复实现意味着每个页面的交互细节、状态行为、留白和响应策略都可能逐渐分叉。

改进目标：

抽出统一的 page chrome 模式，包括标题、搜索、过滤器、顶部轻模块、刷新和空态承接。

建议动作：

把“结构统一、内容可替换”的 page shell 提升成共享组件，而不是继续在每个 home view 内独立写。

预期结果：

首页层体验更一致，同时减少重复代码和视觉漂移。

来源：

[S2-03 原始研究](researches/Stage_3_AR/section_2_shell_ui/03-shared-page-chrome.md)

S2-04 定义 feed/list 状态恢复标准

现状：

当前各页对滚动位置、分页状态、筛选器状态和错误态的处理还不统一。

问题：

用户从详情返回首页、切换 tab、重新进入页面时，可能丢失上下文，或者表现不一致。

改进目标：

统一 feed/list 的状态恢复语义，包括：

1. 滚动位置
2. 筛选状态
3. 已加载分页
4. 空态与错误态
5. 拉取刷新与缓存恢复

建议动作：

抽出共享恢复模型，限制每页自己重新发明状态生命周期。

预期结果：

首页体验从“每页各管各的”变成“一个统一的浏览体验系统”。

来源：

[S2-04 原始研究](researches/Stage_3_AR/section_2_shell_ui/04-feed-state-restoration.md)

S2-05 收敛设计系统边界

现状：

仓库里已经有 DesignTokens、spacing、typography、color token、platform compat，但仍混杂页面局部常量。

问题：

如果设计系统和页面局部实现边界不清，后续维护时会不知道哪些是全局 token，哪些只是某页临时常量。

改进目标：

定义真正的 design system 核心层，并把页面局部视觉决定与系统级 token 分开。

建议动作：

把 spacing、字体、色板、圆角、平台兼容接口的角色重新梳理，防止“页面常量伪装成设计系统”。

预期结果：

设计系统更轻、更稳定，也更容易做全局审计和主题演进。

来源：

[S2-05 原始研究](researches/Stage_3_AR/section_2_shell_ui/05-design-system-boundary.md)

S2-06 重新定义内部工具面

现状：

Features/Infrastructure/ 下已经有 SQLite、OpenClaw、Security、AIMemoryMatching 等诊断/操作页。

问题：

这些页面虽然存在，但还没有清晰的信息架构定位：是开发预览页、内部运维入口、还是将来给运营/高级用户用的工具面，都不明确。

改进目标：

给 internal tools 一个明确身份，定义它们的入口、可见性、权限和与主产品的信息架构关系。

建议动作：

不要把内部工具混在普通用户路径里，也不要让它们永远以“散落调试页”的方式存在。

预期结果：

内部能力可保留、可扩展，但不继续破坏主产品架构纯度。

来源：

[S2-06 原始研究](researches/Stage_3_AR/section_2_shell_ui/06-internal-tools-surface.md)

7. Section 3：Xianxia And Masters Runtime Cleanup

S3-01 拆分 Xianxia 模块边界

现状：

SceneTopicView.swift 同时承载 view、view model、model、repository、configuration、cache 解析等多个层面。

问题：

这种“大而全单文件”虽然起步快，但会让测试、维护、替换网关或扩展缓存策略越来越难。

改进目标：

把 Xianxia 模块按 view、repository、config、cache、parsing 边界拆开。

建议动作：

把运行视图与数据契约、缓存格式、错误模型分离，形成可维护的模块结构。

预期结果：

Xianxia 能从“单文件堆栈”升级为“真正可继续扩展的模块”。

来源：

[S3-01 原始研究](researches/Stage_3_AR/section_3_xianxia_masters/01-xianxia-module-split.md)

S3-02 梳理 Xianxia gateway 契约

现状：

XianxiaTopicRepository 已具备 live 配置、批量拉取、缓存和错误映射，是相对成熟的网关入口。

问题：

如果 gateway contract 不继续正式化，后面会在分页、缓存、失败降级和 schema 兼容上逐步分叉。

改进目标：

明确 topics、shards、cursor、缓存、tenant、错误模型和 transport 层之间的契约。

建议动作：

把 Xianxia gateway 从“实现上能跑”提升到“契约上清楚、可验证、可扩展”。

预期结果：

后面无论是继续接真后端，还是抽 SDK，都更稳。

来源：

[S3-02 原始研究](researches/Stage_3_AR/section_3_xianxia_masters/02-xianxia-gateway-contract.md)

S3-03 修正分页追加语义

现状：

当前 loadMore() 会把 topics/shards 覆盖成最新 batch，而不是追加已有数组。

问题：

这让代码行为和“无限滚动分页”的常识语义不一致，也容易让文档高估当前实现成熟度。

改进目标：

统一 topic/shard 的 append 语义、cursor 前进逻辑和 cache 合并策略。

建议动作：

把分页视作“增量补充”，而不是“拿到下一页就重置现有列表”。

预期结果：

Xianxia 的浏览体验和技术实现将更符合真实分页预期。

来源：

[S3-03 原始研究](researches/Stage_3_AR/section_3_xianxia_masters/03-pagination-append-semantics.md)

S3-04 分解 `MasterExperienceStore`

现状：

MasterExperienceStore.swift 承载目录资产、会话状态、live probe、fallback、ASR、记忆和持久化等多种职责。

问题：

一个 store 吃下太多职责后，任何小改动都容易带来大回归，也难以做清晰测试。

改进目标：

把 masters 模块拆成目录层、会话层、诊断层、远端服务层、fallback 层和本地状态层。

建议动作：

先抽职责边界，再谈继续扩展功能，不再默认让一个 store 继续增肥。

预期结果：

masters 会从“超级 store”演进成“多层协作模块”。

来源：

[S3-04 原始研究](researches/Stage_3_AR/section_3_xianxia_masters/04-master-store-decomposition.md)

S3-05 收敛大师实时对话状态机

现状：

masters 模块已经同时处理 catalog probe / configured candidate / live remote / local fallback / roleplay rewrite。

问题：

如果没有严格状态机和证据格式，这种复杂度很容易在 UI、日志、自动化和用户感知之间失配。

改进目标：

把 masters live chat 定义成一个明确状态机，而不是若干 if/else 的累积。

建议动作：

统一页面状态、自动化 smoke 状态、结果文件状态和用户可见 blocker。

预期结果：

用户、开发者和自动化都能用同一语言描述“当前到底接没接通 live”。

来源：

[S3-05 原始研究](researches/Stage_3_AR/section_3_xianxia_masters/05-master-live-chat-state-machine.md)

S3-06 重构 ASR readiness 契约

现状：

ASR 已有比较强的配置诊断和 blocker 机制，但仍受 endpoint、auth、preview host 能力和真实 live 可用性约束。

问题：

ASR 最容易出现“代码支持了，但环境没配”“页面看起来能点，但其实不能用”的错觉。

改进目标：

把 ASR readiness 拆成独立契约：

1. endpoint readiness
2. auth readiness
3. host capability readiness
4. smoke validation readiness
5. blocker reporting

建议动作：

继续坚持“没 ready 就别假装 ready”的原则，把 blocker 精准前置。

预期结果：

ASR 链路会更诚实，也更容易联调。

来源：

[S3-06 原始研究](researches/Stage_3_AR/section_3_xianxia_masters/06-master-asr-readiness.md)

8. Section 4：EarnSocial, Messages, And Profile Consistency

S4-01 确定 EarnSocial 单一运行真相

现状：

当前首页由 EarnSocialHomeView.swift 的本地 fixture 主导，而大型 EarnSocialExperienceStore.swift 仍存在但不是首页运行真相。

问题：

这是典型双路径：一个是当前真实运行路径，一个是更复杂但未真正接线的经验层。

改进目标：

必须决定谁是主路径，谁是支持/未来路径，不能继续让两者都像“主实现”。

建议动作：

优先围绕当前真实首页做治理，把 ExperienceStore 的角色重新定义清楚。

预期结果：

EarnSocial 不再给人“好像已经 fully wired”的错觉。

来源：

[S4-01

原始研究](researches/Stage_3_AR/section_4_social_messages_profile/01-earn-social-single-runtime-path.md)

S4-02 治理 EarnSocial fixture 与 assets

现状：

首页卡片、偏好标签、聊天 seed 等大量内容写在页面文件内部。

问题：

内容写死在页面里，会让数据治理、测试、未来 live 对接和设计修改都很痛苦。

改进目标：

把 fixture、seed 和 assets 引用外提成可治理的数据源层。

建议动作：

优先把 live 首页需要的卡片数据、偏好数据、聊天 seed 从页面里移出去，再逐步对齐 ExperienceStore。

预期结果：

EarnSocial 的内容层会更接近真正的数据源，而不是“把世界观写在 Swift 文件里”。

来源：

[S4-02

原始研究](researches/Stage_3_AR/section_4_social_messages_profile/02-earn-social-fixture-governance.md)

S4-03 梳理消息模块真实领域边界

现状：

消息模块包含 IM hub、thread、mask、relationship、memory、group play 等多种能力。

问题：

如果继续把这些混在少数几个 UI/Store

文件里，后面会越来越难区分“首页模型”“会话模型”“关系模型”“玩法模型”。

改进目标：

给消息模块画出清晰的领域边界。

建议动作：

把 home、thread、mask、relationship、memory、group play 分成更清晰的子域。

预期结果：

消息页不再是“一个功能大杂烩”，而是“一个清晰的熟人关系域”。

来源：

[S4-03 原始研究](researches/Stage_3_AR/section_4_social_messages_profile/03-messages-domain-boundary.md)

S4-04 整合消息模块导航面

现状：

消息模块虽然已有多种高级子页，但它们在导航层的集成还不够体系化。

问题：

存在“组件在仓库里，但用户路径不明确”的现象。

改进目标：

让这些高级子页拥有清晰入口、返回路径、状态承接和导航关系。

建议动作：

以 IM hub 为入口，把 thread、关系页、mask 页、group play 等放入明确导航树。

预期结果：

消息模块能从“若干孤立页面”演进成“一个结构明确的子产品”。

来源：

[S4-04 原始研究](researches/Stage_3_AR/section_4_social_messages_profile/04-messages-surface-integration.md)

S4-05 统一 MyProfile 根页数据来源层级

现状：

MyProfileView.swift 根页仍以 mock 数据为主，但旁边已经有更真实的 MyProfileOverviewMetrics 提供者。

问题：

如果根页不明确自己的数据层级，就会一直停留在“视图已经很好看，但数据来源不诚实”的状态。

改进目标：

定义根页数据来源优先级：哪些指标走 live-ish provider，哪些仍保留 mock，哪些必须标注来源。

建议动作：

优先把可真实获取的指标接进根页，再把仍属 mock 的部分清晰标记为 mock。

预期结果：

我的页会从“视觉卡片集合”升级为“带真实数据层次的控制台首页”。

来源：

[S4-05 原始研究](researches/Stage_3_AR/section_4_social_messages_profile/05-my-profile-data-provenance.md)

S4-06 定义跨 tab handoff 契约

现状：

当前产品强依赖跨 tab handoff，例如 xianxia -> earn social、masters -> messages/profile 等，但状态载荷还没有统一契约。

问题：

没有统一 handoff contract 时，跨 tab 跳转会逐渐依赖临时参数拼装，最终难维护、难恢复、难测。

改进目标：

定义跨 tab route 和 state payload 的统一契约。

建议动作：

把 route key、payload schema、恢复语义和来源模块都标准化。

预期结果：

跨 tab 行为不再是“勉强能跳”，而是“有语义、有恢复能力的 handoff”。

来源：

[S4-06 原始研究](researches/Stage_3_AR/section_4_social_messages_profile/06-cross-tab-handoff-contract.md)

9. Section 5：Repo Boundaries, Validation, And Automation

S5-01 Support code 角色分类

现状：

.mjs 服务、LocalBackend、Domain/UseCases、plugin 代码和 Swift runtime 共仓。

问题：

如果不区分 shipped support code、prototype、contract、future lane scaffolding，就会一直误判真实运行面。

改进目标：

给 support code 明确角色分类和边界说明。

建议动作：

不是删除 support code，而是把它放回正确角色位置。

预期结果：

仓库会更容易判断“什么是主路径，什么是支持路径，什么是未来路径”。

来源：

[S5-01

原始研究](researches/Stage_3_AR/section_5_boundaries_validation/01-support-code-role-classification.md)

S5-02 对齐 plugin 与 iOS app 契约

现状：

spare-life-openclaw-plugin 已经是独立 workspace，并对 topic gateway 和 SDK 产生实际影响。

问题：

如果 app 和 plugin 不共享明确契约，schema 和 demo 很容易逐渐漂移。

改进目标：

定义 plugin runtime、SDK、payload schema、gateway 和 iOS app 之间的同步规则。

建议动作：

把契约变化看作跨工作区变更，而不是各自局部修改。

预期结果：

plugin 和 app 会更像“同一系统的两个边界”，而不是“放在同一仓库里的两个项目”。

来源：

[S5-02 原始研究](researches/Stage_3_AR/section_5_boundaries_validation/02-plugin-app-contract-sync.md)

S5-03 审计 `SpareLifeCore` package boundary

现状：

SpareLifeCore 目前只纳入 Swift runtime 相关目录，support.mjs、plugin、local backend 并未进入 package。

问题：

如果 package boundary 不持续治理，后面很容易把不该进入 package 的东西塞进去，或者误以为某些 support code 已经成为运行时依赖。

改进目标：

明确 package 该承载什么，不该承载什么。

建议动作：

以“Swift runtime 可复用核心”为 package 边界，而不是“仓库里有什么就塞什么”。

预期结果：

package 结构会更稳定，也更利于测试和未来拆分。

来源：

[S5-03 原始研究](researches/Stage_3_AR/section_5_boundaries_validation/03-package-boundary-audit.md)

S5-04 建立统一验证矩阵

现状：

当前验证手段分散在 Swift tests、UI tests、plugin demos、gateway probes 和文档证据里。

问题：

如果没有矩阵，就无法回答“一个能力到底由什么验证、在哪一级验证、缺哪一级验证”。

改进目标：

建立覆盖 app shell、module runtime、gateway、plugin、docs evidence 的统一验证矩阵。

建议动作：

把验证按层次组织，而不是按个人习惯零散记录。

预期结果：

验证从“有很多测试”升级为“有结构的验证体系”。

来源：

[S5-04 原始研究](researches/Stage_3_AR/section_5_boundaries_validation/04-validation-matrix.md)

S5-05 全仓 fixture 与 seed 数据治理

现状：

当前仓库至少同时存在页面级 fixture、store mock、support seed、plugin fixture、demo seed 等多套世界观。

问题：

不同模块各自维护 fixture，会导致人物、分类、卡片、群组、状态和文案世界观不一致。

改进目标：

建立统一的 fixture/seed 治理规则，至少做到“来源可登记、语义可映射、主路径可优先”。

建议动作：

优先治理当前 live runtime 直接使用的 fixture，再逐步收敛 support/demo 层的 seed。

预期结果：

fixture 不再只是“开发方便写的假数据”，而是“有边界、有语义的一类仓库资产”。

来源：

[S5-05 原始研究](researches/Stage_3_AR/section_5_boundaries_validation/05-fixture-seed-governance.md)

S5-06 建立 Stage 3 自动化运行规约

现状：

Stage 3 已经拥有自己的 automation clone、slot worker、guard、cron、日志和回写机制。

问题：

如果自动化本身没有治理规则，它很快会从“治理工具”变成“新的混乱源”。

改进目标：

把 automation clone、worker ownership、guard merge、cleanup、operator handoff 和日志语义全部定义成规则。

建议动作：

继续把自动化当成正式的仓库能力治理，而不是一次性脚本。

预期结果：

以后类似的 Stage 4、专题优化或大规模研究批次，可以复用这套方式而不是重新搭架子。

来源：

[S5-06 原始研究](researches/Stage_3_AR/section_5_boundaries_validation/06-automation-governance.md)

10. 总体优先级排序

如果把 30 个方向压缩成真正的执行优先级，建议按下面顺序理解：

P0：必须先守住的真相层

1. S1-01 命名词典
2. S1-02 文档分层
3. S1-04 运行真相地图
4. S1-05 配置来源登记
5. S1-06 验证证据格式

P1：必须先稳住的应用壳和主运行链路

1. S2-01 壳层元数据
2. S2-02 导航契约
3. S2-03 page chrome

4. S2-04 state restoration
5. S3-01 Xianxia 模块拆边界
6. S3-04 Master store 分层
7. S3-05 live chat 状态机
8. S3-06 ASR readiness

P2：需要尽快收敛的双路径和 mock/live 漂移

1. S4-01 EarnSocial 单一路径
2. S4-02 EarnSocial fixture 治理
3. S4-03 Messages 子域边界
4. S4-05 Profile 根页数据来源
5. S5-05 fixture/seed 全仓治理

P3：支撑长期扩展的仓库级能力

1. S5-01 support code 分类
2. S5-02 plugin-app 契约同步
3. S5-03 package boundary
4. S5-04 验证矩阵
5. S5-06 自动化规约

11. Stage 3 最重要的 10 个产出

如果只看最关键结果，Stage 3 实际交付了下面 10 件事：

1. 明确了“代码优先，旧文档服从代码”的解释顺序。
2. 明确了这个仓库不是纯 iOS app，而是混合仓库。
3. 明确了 Xianxia 与 Masters 是最接近真实链路的主模块。
4. 明确了 EarnSocial、Messages、Profile 的根页仍存在 mock/live 或双路径问题。
5. 明确了命名、文档、配置、验证四大真相治理规则。
6. 明确了 app shell、shared UI、navigation 和 internal tools 的壳层边界。
7. 明确了 Xianxia 分页语义、Masters store 分层、live chat 状态机和 ASR readiness 契约。
8. 明确了跨 tab handoff、fixture 治理和 profile 根页数据来源。
9. 明确了 support code、plugin、package 和验证矩阵的仓库级边界。
10. 把这套治理方式变成了可重复执行的自动化流程，而不是一次性人工整理。

12. 后续建议

Stage 3 结束之后，下一阶段不要立刻继续新增功能，而应该先做一轮“基于 Stage 3 结论的代码收敛实现”。建议顺序如下：

1. 先收壳层：

统一 app shell metadata、navigation contract 和 page chrome。

2. 再收主链路：

拆分 Xianxia 与 Masters 的大文件/大 store，修正分页和状态机。

3. 再收双路径：

处理 EarnSocial、Messages、Profile 的 runtime truth 与 fixture 治理。

4. 最后收仓库边界：

把 support code、package、plugin、validation matrix 和 automation 常态化。

13. 关联文档

- [Stage 3 审计](Stage_3_Codebase_Audit.md)
- [Stage 3 蓝图](Stage_3_AR_Blueprint.md)
- [Stage 3 研究总目录](researches/Stage_3_AR/README.md)

原始单项研究文档共 30 篇，全部位于 Docs/researches/Stage_3_AR/ 下，可按 section 继续深挖。